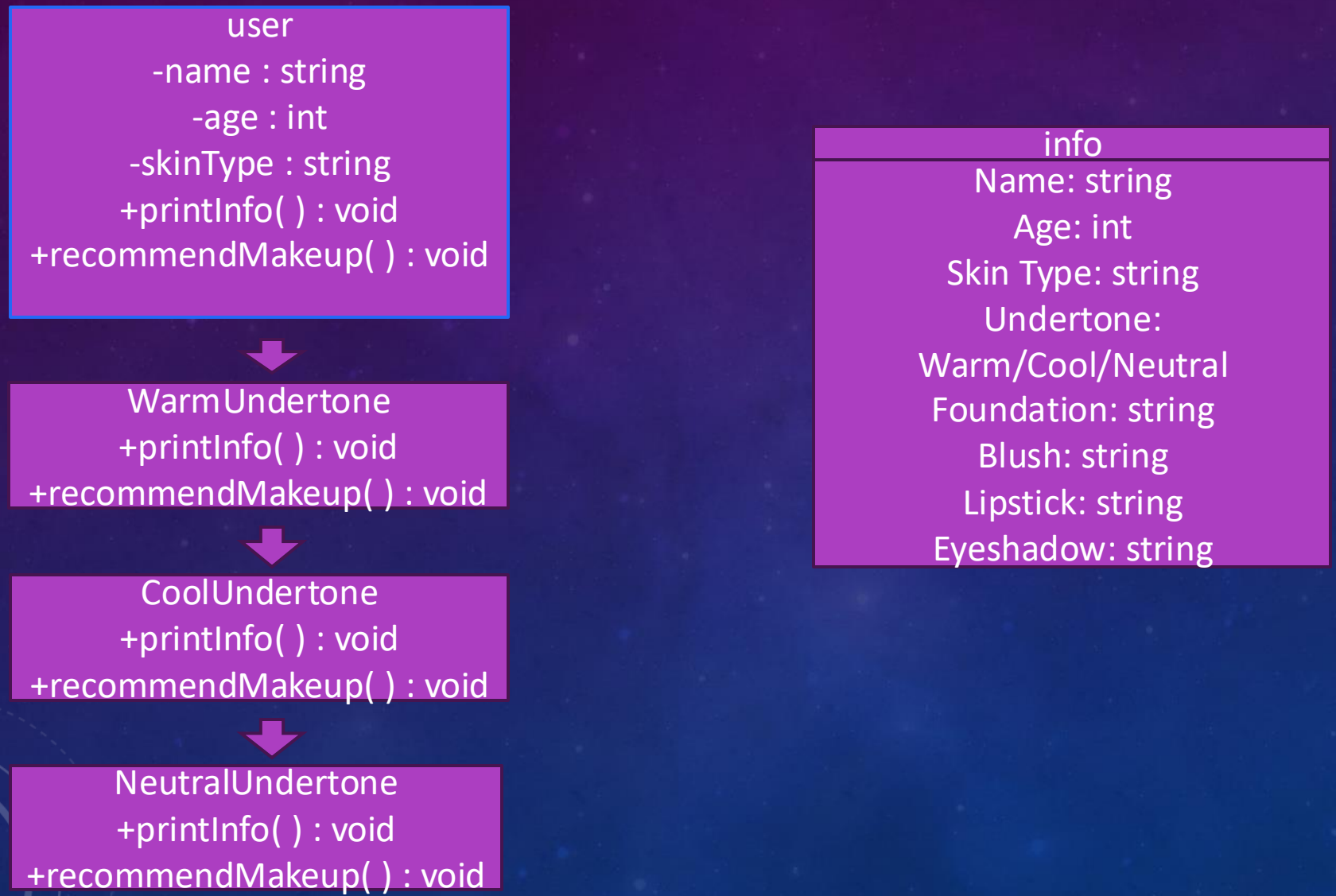
MAKEUP RECOMMENDATIONS SYSTEM

- Işıl hardal
- Irmak nisa şahin
- Şevval yüksek

PROJECT SUBJECT

- This project is a Makeup Recommendation System developed using Object-Oriented Programming principles in C++.
- The system collects user information such as name, age, skin type and skin undertone. According to the user's undertone (warm, cool or neutral) the program recommends suitable makeup products including foundation, blush, lipstick and eyeshadow colors
- The main purpose of the project is to demonstrate Object-Oriented Programming (OOP) concepts such as Inheritance, Polymorphism, Virtual Functions, Encapsulation.
- The project uses a base class called User and derived classes. They are WarmUndertone , CoolUndertone , NeutralUndertone .
- Each derived class overrides the recommendMakeup() function to provide personalized makeup recommendations.

UML DIAGRAMS



SCREENSHOTS/EXPLANATION

- We added the library just because we used setw(). This allows us to set field widths and keep the console output neatly aligned in columns.
- We defined a base class named User with protected attributes. The variables name, age, and skinType are kept protected so that they can be directly accessed and inherited by derived (child) classes.
- The printInfo() function is defined as a virtual function. This function prints the formatted user information. Making it virtual allows derived classes to override it and display specialized data using polymorphism.
- We implemented a pure virtual function named recommendMakeup() = 0. This makes User an abstract class. It cannot be instantiated directly, and any class inheriting from User must provide its own specific implementation for this function.
- A virtual destructor is defined. This is crucial in polymorphism to ensure that when a derived class object is deleted through a base class pointer, the derived class's destructor is properly called, preventing memory leaks.

```cpp
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

// Base Class
class User {
protected:
    string name;
    int age;
    string skinType;

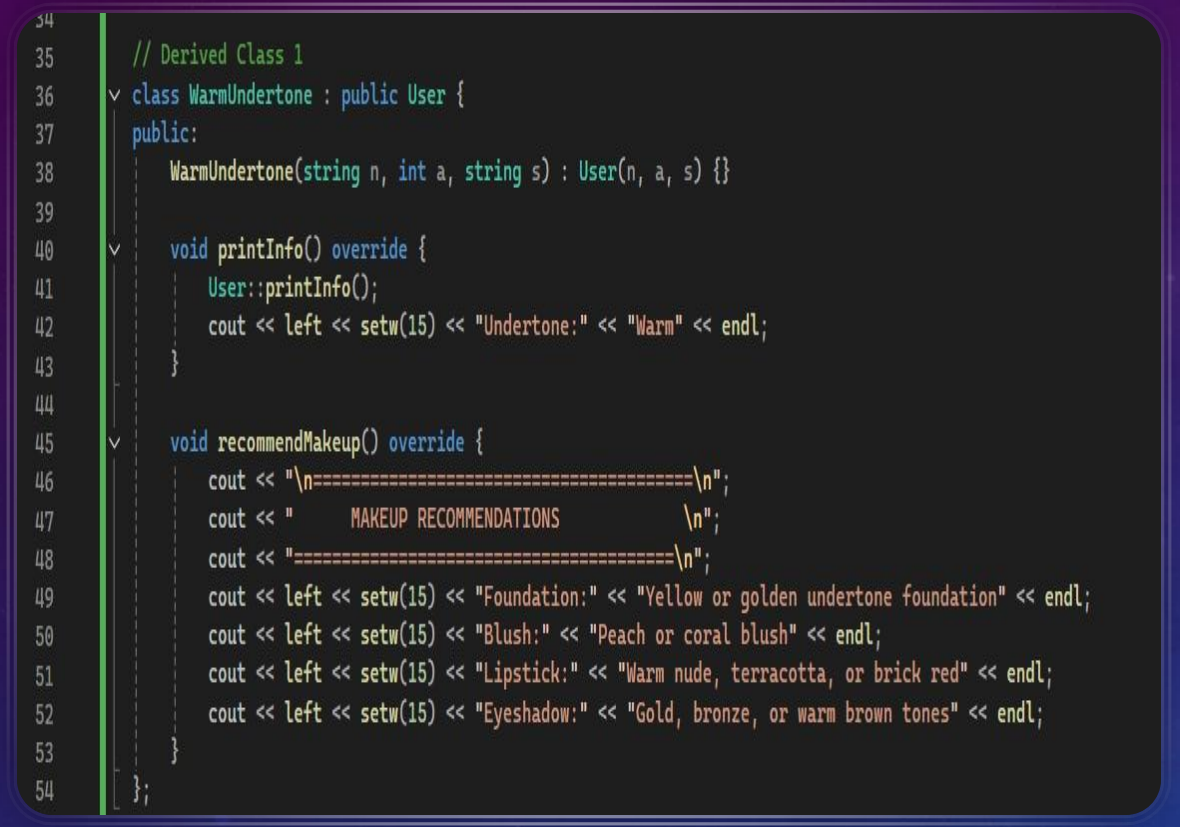
public:
    User(string n, int a, string s) {
        name = n;
        age = a;
        skinType = s;
    }

    virtual void printInfo() {
        cout << "\n=======================================\n";
        cout << "              USER INFORMATION          \n";
        cout << "=======================================\n";
        cout << left << setw(15) << "Name:" << name << endl;
        cout << left << setw(15) << "Age:" << age << endl;
        cout << left << setw(15) << "Skin Type:" << skinType << endl;
    }

    // Pure virtual function for polymorphism
    virtual void recommendMakeup() = 0;

    virtual ~User() {}
};
```

SCREENSHOTS/EXPLANATION



```
// Derived Class 1
class WarmUndertone : public User {
public:
    WarmUndertone(string n, int a, string s) : User(n, a, s) {}

    void printInfo() override {
        User::printInfo();
        cout << left << setw(15) << "Undertone:" << "Warm" << endl;
    }

    void recommendMakeup() override {
        cout << "\n===================================\n";
        cout << "      MAKEUP RECOMMENDATIONS          \n";
        cout << "===================================\n";
        cout << left << setw(15) << "Foundation:" << "Yellow or golden undertone foundation" << endl;
        cout << left << setw(15) << "Blush:" << "Peach or coral blush" << endl;
        cout << left << setw(15) << "Lipstick:" << "Warm nude, terracotta, or brick red" << endl;
        cout << left << setw(15) << "Eyeshadow:" << "Gold, bronze, or warm brown tones" << endl;
    }
};
```

•Derived Class 1: WarmUndertone

- We created a derived class named WarmUndertone that publicly inherits from the User base class.
- The constructor of WarmUndertone takes name, age, and skin type as parameters and directly passes them to the base class constructor using a member initializer list.
- The printInfo() function is overridden to first call the base class function using User::printInfo(), and then it adds specific warm undertone details to the console output.
- The recommendMakeup() function overrides the pure virtual function from the base class, providing customized makeup suggestions specifically tailored for users with warm undertones.

SCREENSHOTS/EXPLANATION

```
// Derived Class 2
class CoolUndertone : public User {
public:
    CoolUndertone(string n, int a, string s) : User(n, a, s) {}

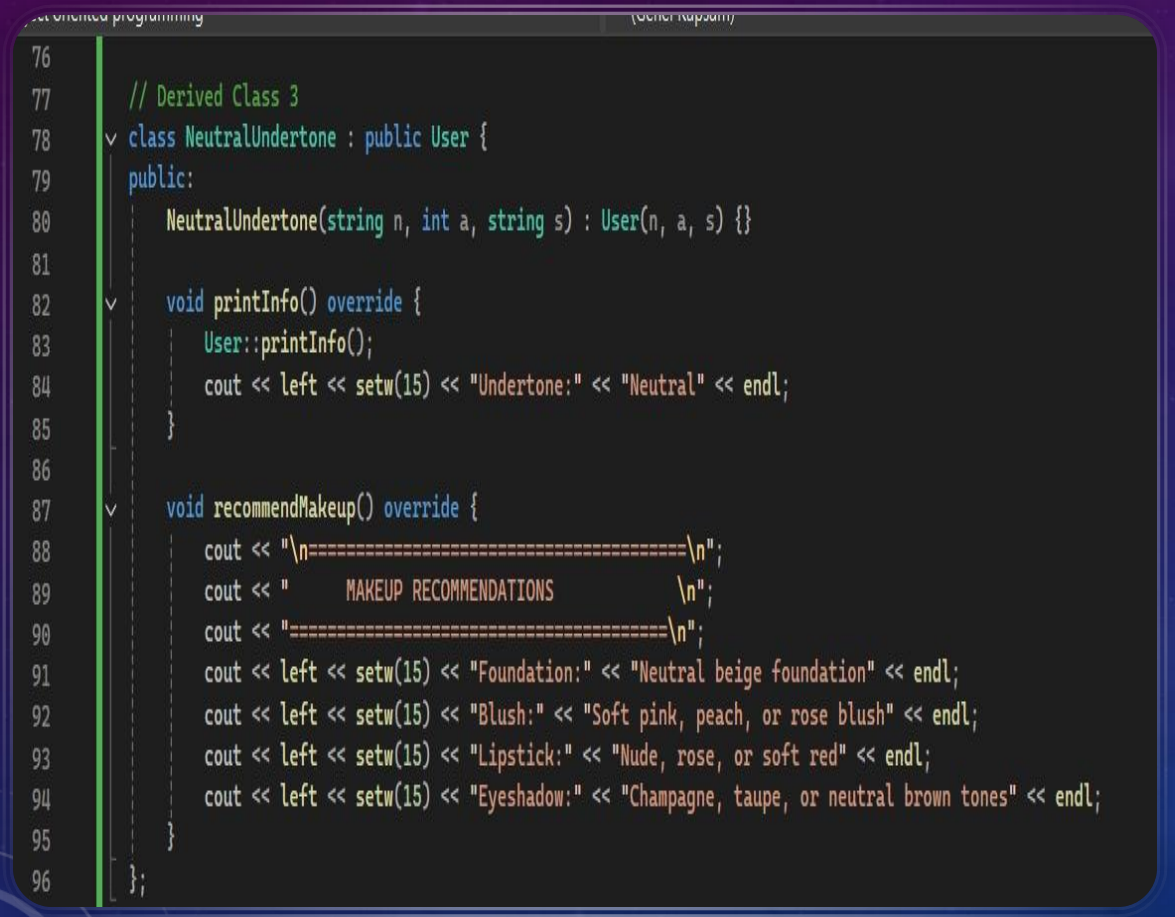
    void printInfo() override {
        User::printInfo();
        cout << left << setw(15) << "Undertone:" << "Cool" << endl;
    }

    void recommendMakeup() override {
        cout << "\n=====================================\n";
        cout << "      MAKEUP RECOMMENDATIONS           \n";
        cout << "=====================================\n";
        cout << left << setw(15) << "Foundation:" << "Pink or rosy undertone foundation" << endl;
        cout << left << setw(15) << "Blush:" << "Pink or berry blush" << endl;
        cout << left << setw(15) << "Lipstick:" << "Rose, berry, or blue-based red" << endl;
        cout << left << setw(15) << "Eyeshadow:" << "Silver, mauve, or cool brown tones" << endl;
    }
};
```

•Derived Class 2: CoolUndertone

- We created a second derived class named CoolUndertone that publicly inherits from the User base class.
- The constructor utilizes the base class constructor to handle initialization of inherited member variables seamlessly.
- The printInfo() function uses overriding and the scope resolution operator to extend the base class output by appending cool undertone specifications.
- The recommendMakeup() function overrides the pure virtual function to supply product recommendations optimal for cool undertones, such as pink-based foundation and silver shades.

SCREENSHOTS/EXPLANATION



```
76
77  // Derived Class 3
78  class NeutralUndertone : public User {
79  public:
80      NeutralUndertone(string n, int a, string s) : User(n, a, s) {}
81
82      void printInfo() override {
83          User::printInfo();
84          cout << left << setw(15) << "Undertone:" << "Neutral" << endl;
85      }
86
87      void recommendMakeup() override {
88          cout << "\n=====================================\n";
89          cout << "      MAKEUP RECOMMENDATIONS           \n";
90          cout << "=====================================\n";
91          cout << left << setw(15) << "Foundation:" << "Neutral beige foundation" << endl;
92          cout << left << setw(15) << "Blush:" << "Soft pink, peach, or rose blush" << endl;
93          cout << left << setw(15) << "Lipstick:" << "Nude, rose, or soft red" << endl;
94          cout << left << setw(15) << "Eyeshadow:" << "Champagne, taupe, or neutral brown tones" << endl;
95      }
96  };
```

•Derived Class 3: NeutralUndertone

- We created a third derived class named NeutralUndertone that completes our polymorphic setup by publicly inheriting from User.
- The constructor passes the initialization variables up to the base class constructor in exactly the same structured manner as the other sub-classes.
- The printInfo() function is overridden to invoke the base behavior and cleanly append neutral undertone specifics to the data display.
- The recommendMakeup() function fulfills the contract of the abstract base class by implementing tailored makeup recommendations suited exclusively for neutral skin undertones.

```
98  int main() {
99      string name, skinType;
100     int age;
101     int choice;
102
103     cout << "Enter your name: ";
104     getline(cin, name);
105
106     cout << "Enter your age: ";
107     cin >> age;
108
109     cout << "Enter your skin type ";
110     cout << "(dry / oily / combination): ";
111     cin >> skinType;
112
113     cout << "\nChoose your skin undertone:\n";
114     cout << "1. Warm\n";
115     cout << "2. Cool\n";
116     cout << "3. Neutral\n";
117     cout << "Your choice: ";
118     cin >> choice;
119
120     User* user = nullptr;
```

SCREENSHOTS/EXPLANATION

•We defined local variables inside the main function to store the user input for name, age, skin type, and menu choice.

- We used the getline() function to read the full name string including spaces, and standard cin for numeric and single-word inputs.
- We implemented an interactive menu display that prompts the user to select their specific skin undertone from three available options.
- We declared a base class pointer named user and initialized it to nullptr. This pointer is essential for achieving runtime polymorphism later in the code.

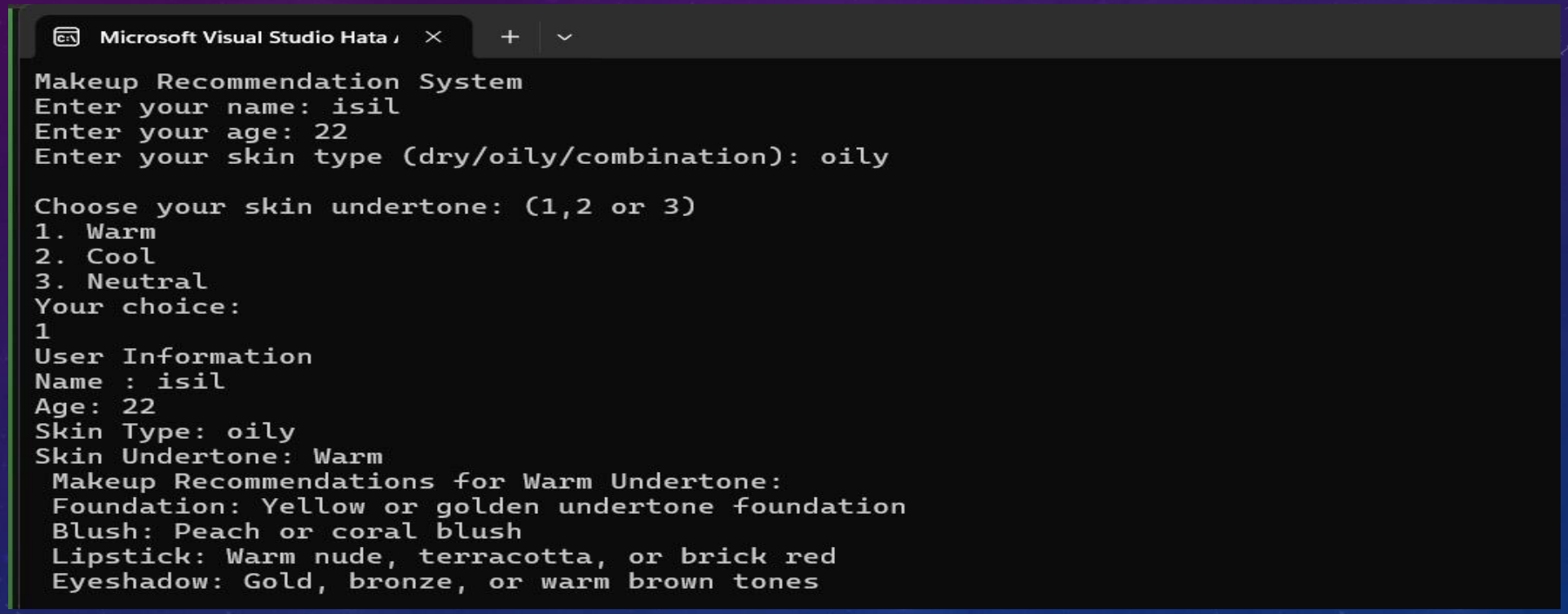
SCREENSHOTS/EXPLANATION

```
121
122     if (choice == 1) {
123         user = new WarmUndertone(name, age, skinType);
124     }
125     else if (choice == 2) {
126         user = new CoolUndertone(name, age, skinType);
127     }
128     else if (choice == 3) {
129         user = new NeutralUndertone(name, age, skinType);
130     }
131     else {
132         cout << "\nInvalid choice. Program ended.\n";
133         return 0;
134     }
135
136     user->printInfo();
137     user->recommendMakeup();
138
139     delete user;
140
141     return 0;
142 }
143
```

•We implemented an if-else structure to evaluate the user's choice and dynamically allocate memory for the corresponding derived class object using the new keyword.

- We included a fallback else block to catch any invalid input options, printing an error message and safely terminating the program early.
- We called printInfo() and recommendMakeup() through the base class pointer user, demonstrating runtime polymorphism as the correct derived class functions are executed dynamically.
- We used the delete keyword to free the allocated memory from the heap, ensuring proper cleanup via the virtual destructor and preventing memory leaks before the program returns zero.

OUTPUT


Microsoft Visual Studio Hata
Makeup Recommendation System
Enter your name: isil
Enter your age: 22
Enter your skin type (dry/oily/combination): oily
Choose your skin undertone: (1,2 or 3)
1. Warm
2. Cool
3. Neutral
Your choice:
1
User Information
Name : isil
Age: 22
Skin Type: oily
Skin Undertone: Warm
Makeup Recommendations for Warm Undertone:
Foundation: Yellow or golden undertone foundation
Blush: Peach or coral blush
Lipstick: Warm nude, terracotta, or brick red
Eyeshadow: Gold, bronze, or warm brown tones